

CONTENTS

- Overview
- 1 The bottleneck is attention, not intelligence
- 2 The five multi-agent communication patterns
- 3 Missions: four patterns in one workflow, three roles
- 4 Validation contracts: define 'done' before writing code
- 5 Two validators, adversarial by design
- 6 Structured handoffs and self-healing
- 7 Serial execution beats parallelism
- 8 The right model in each seat
- 9 Beating the bitter lesson: prompts, not state machines
- 10 Production reality, and what it unlocks
- Glossary
- Check yourself
- Where to go next

Multi-Agent Architectures That Ship: Five Patterns, Three Roles & Validation Contracts

Assemble agent teams that run for days — by separating who builds from who verifies, defining 'done' before any code exists, and putting the right model in each seat.

For engineers who have built single-agent workflows and now want agent teams that stay coherent over hours or days · 30 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- You have built an agent that calls tools in a loop
- Familiarity with sub-agents and why context windows fill up
- Comfort with git-based workflows and CI concepts (tests, linting, type checking)

Overview

The bottleneck in software engineering is no longer intelligence — it is human attention. A team may have fifty features in the backlog and models capable of implementing all of them, but only enough bandwidth to supervise a few per day, because every task needs attention and every commit needs review. The interesting question becomes: what if a human decides what to build, and a system figures out how?

Multi-agent research is a mess of competing frameworks and terminology, so this lesson starts with a taxonomy that cuts through it — five communication patterns from which every multi-agent system is assembled. It then walks through one production architecture built from four of them, called missions, whose longest run lasted sixteen days.

The surprises are where the value is. Parallelism, the obvious way to go faster, made things worse and was abandoned for serial execution with narrow internal parallelism. Tests written by the agent that wrote the code turn out to confirm decisions rather than catch bugs, which forced validation to be redesigned around a contract written before implementation. And the whole orchestration lives in roughly 700 lines of prompt text rather than a state machine, specifically so the system improves when models improve instead of being obsoleted by them.

KEY TAKEAWAYS

- ✓ Five patterns compose every multi-agent system: delegation, creator-verifier, direct communication, negotiation, and broadcast.
- ✓ Direct communication (agents DMing each other with no coordinator) is the one to avoid — state fragments and there is no single source of truth.
- ✓ Separate the agent that builds from the agent that verifies: the author has a cost bias toward its own code working, exactly why humans do code review.
- ✓ A validation contract, written during planning before any code, defines correctness independently of implementation — often hundreds of assertions, each feature owning one or more.
- ✓ Tests written after implementation are shaped by the code rather than the intent, so they confirm decisions instead of catching bugs.
- ✓ Validate behaviour, not just code: a QA-style validator drives the running application through computer use, and that real-world execution — not token generation — consumes most of a mission's wall-clock time.
- ✓ Structured handoffs (what was done, what was left, commands run with exit codes, issues found) are what let the system self-heal at milestone boundaries.
- ✓ Serial execution with targeted read-only parallelism beat parallel workers: coordination overhead ate the speed gains while the error rate rose.
- ✓ Planning, implementation, and validation reward different model strengths, and no single provider is best at all three — being model-agnostic means you are not capped by one family's weakest capability.
- ✓ Keep orchestration in prompts and skills rather than a hard-coded state machine, so every model improvement makes the system better instead of obsolete.

01 The bottleneck is attention, not intelligence

0:47

Models can already handle the whole backlog; what runs out is the human bandwidth to supervise them.

The framing that motivates everything else: today's models are smart enough to work through fifty backlog features, but every task requires someone's attention and every commit requires someone's review. Even excellent engineers drive only a couple of tasks forward at once, so throughput is capped by supervision rather than capability.

That reframes the goal. Instead of making an agent smarter, you make a system that can be trusted to run unattended — a human decides **what** to build, and the system works out **how**, for hours or days, while the human does something else.

The constraint this imposes is severe and it shapes every later design decision. A system that needs a human to notice when it goes wrong has not removed the bottleneck at all; it has just moved it. So the architecture has to detect its own failures, correct them, and preserve enough context across many hours that it does not drift. Every mechanism in this lesson exists to satisfy that requirement.

KEY POINTS

- A team can have fifty implementable features and still ship a few per day, because supervision is the scarce resource.
- Every task needs attention and every commit needs review — that is what caps throughput, not model capability.
- The goal shifts from a smarter agent to a system trustworthy enough to run unattended.
- A system that requires a human to spot its mistakes has moved the bottleneck rather than removed it.

Where the ceiling actually sits

Fifty features, each of which a model could implement in an hour. With one engineer reviewing, throughput is bounded by how many changes that engineer can meaningfully read in a day — perhaps three or four. Adding a second agent doubles the work produced and doubles the review load, so nothing improves until verification itself is delegated.

02 The five multi-agent communication patterns

1:28

Delegation, creator-verifier, direct communication, negotiation, and broadcast — the primitives every framework is built from.

The multi-agent field is fragmented: everyone has their own framework, terminology, and opinions about what works. A simple taxonomy cuts through it, because every system is assembled from five communication patterns.

****Delegation**** is one agent spawning another and receiving a result — a parent saying 'go figure out the database schema' and getting an answer back. It is the simplest form of multi-agent communication and what almost everyone implements first; sub-agents in coding tools are the common example.

****Creator-verifier**** has one agent build something and a second check it. The point is separation of concerns: the agent that wrote the code carries a cost bias, because it wants that code to work. A fresh agent with fresh context is far more likely to find real issues — the same reason humans review each other's code rather than their own.

****Direct communication**** is agents talking without a central coordinator, effectively DMin'g each other. It is hard to get right, because state fragments across those conversations and there is no single source of truth.

****Negotiation**** is agents communicating over a shared resource — the same API, the same region of the codebase. It need not be adversarial; the best case is net positive-sum trading, where interacting agents find a genuine win-win.

****Broadcast**** is one agent informing many: status updates, new context that applies to everyone, shared constraints. It is the least flashy pattern and it is critical for keeping a long-running task coherent.

KEY POINTS

- Delegation: an agent spawns another and gets a result back. Simplest pattern, usually implemented first, e.g. sub-agents.
- Creator-verifier: builder and checker are separate, because the author is cost-biased toward its own work succeeding.
- Direct communication: no coordinator, agents message each other. Hard to get right — state fragments, no single source of truth.
- Negotiation: coordination over a shared resource, ideally positive-sum rather than adversarial.
- Broadcast: one-to-many information sharing. Unglamorous but essential to coherence over long runs.

The same task under each pattern

One feature — 'add rate limiting to the public API' — routed through each primitive. Notice that four of them compose cleanly while direct communication is the one that loses track of state.

delegation	orchestrator -> worker: "implement rate limiting" worker -> researcher: "what limiter does this repo use?"
creator-verifier	worker writes it; a fresh validator checks it (the validator has never seen the code before)
broadcast	orchestrator -> all: "redis is now a hard dependency"
negotiation	two features both touch middleware/auth.py -> resolved at a milestone boundary, not by a race
direct comms	worker A <-> worker B agree an interface privately -> nobody else knows; the shared state is now wrong

Orchestrator plans, workers implement, validators verify — an ecosystem communicating through structured handoffs and shared state, not one long agent session.

Missions composes four of the five patterns — delegation, creator-verifier, broadcast, and negotiation — into a single workflow. Notably it omits direct communication, for the reason given above. You describe a goal, scope it through conversation, approve a plan, and the system executes for hours or days.

The crucial distinction is that a mission is not a single agent session. It is an ecosystem of agents communicating through structured handoffs and shared state, organised into three roles.

The **orchestrator** handles planning. When you describe what you want it acts as a sounding board, asks strategic questions, and probes unclear requirements. It produces a plan containing features, milestones, and a validation contract.

Workers handle implementation. A worker assigned a feature starts with clean context — no accumulated baggage, no degraded attention. It reads its spec, implements the feature, and commits via git, so the next worker inherits both a clean slate and a working codebase. That per-feature reset is what stops quality decaying as a run gets longer.

Validators handle verification. Most systems validate by running lint, type checks, and tests, perhaps with code review. Missions does all of that and also validates behaviour — not 'does the code look right?' but 'does this work end to end?' That is the difference that lets a run continue for days without drifting.

KEY POINTS

- Missions composes delegation, creator-verifier, broadcast and negotiation — deliberately excluding direct communication.
- You describe a goal, scope it conversationally, approve a plan; the system then executes for hours or days.
- A mission is an ecosystem communicating via structured handoffs and shared state, not one long session.
- Orchestrator: plans, interrogates requirements, and emits features, milestones and a validation contract.
- Workers: one feature each with clean context, then commit via git so the next worker inherits a working codebase.
- Validators: lint/types/tests and code review, plus end-to-end behavioural verification.

The shape of a mission plan

The orchestrator's output. Features map onto milestones, and every feature carries the assertions it is responsible for satisfying.

```
mission: "build a Slack-style team chat"

milestone 1  auth + workspaces
  F1  email/password signup      -> asserts A1, A2
  F2  workspace creation         -> asserts A3
  F3  invite by email           -> asserts A4, A5

milestone 2  channels + messaging
  F4  create/join channels       -> asserts A6, A7
  F5  post + render messages     -> asserts A8, A9, A10
  ...

validation_contract: A1..A147  (written BEFORE any code)
invariant: every assertion is covered by >= 1 feature
```

Why a worker gets fresh context

A single agent session that implements twenty features carries the residue of all twenty — earlier dead ends, abandoned approaches, stale file contents — and its attention degrades as the window fills. Giving each feature a new worker whose only inputs are the spec and the committed codebase means feature twenty is implemented as carefully as feature one.

04 Validation contracts: define 'done' before writing code

6:15

Tests written after implementation are shaped by the code rather than the intent, so they confirm decisions instead of catching bugs.

This is the sharpest insight in the talk. You have probably seen the pattern: an agent builds a feature, writes tests, the tests pass, coverage is high — and the feature is still wrong. The reason is that the tests were shaped by the code, not by what the code was supposed to do. Stated plainly: **tests written after implementation don't catch bugs, they confirm decisions.** Build a system that validates itself that way and it will eventually drift, with a green suite the whole time.

The validation contract breaks that circularity by ordering. It is written during planning, before any code exists, and it defines correctness independently of implementation. For a complex project it can run to hundreds of assertions. Each feature is then assigned one or more assertions it must satisfy, with the invariant that the sum of all features covers every assertion.

Because the contract predates the code, no implementation choice can quietly redefine what counts as correct. An agent cannot weaken an assertion to make its work pass, the way it can weaken a test it also wrote. And because assertions are allocated to features up front, a feature that satisfies its own assertions but breaks another's is caught rather than absorbed.

Write assertions about observable behaviour, not internal structure. 'Posting a message makes it appear for every member of that channel without a reload' survives refactors; 'MessageService.post() calls broadcast()' does not.

KEY POINTS

- Tests written after implementation are shaped by the code, so they confirm decisions rather than catching bugs.
- The validation contract is written during planning, before any code, and defines correctness independently of implementation.
- A complex project may have hundreds of assertions; each feature is assigned one or more.
- The invariant: the sum of all features must cover every assertion in the contract.
- Because the contract predates the code, an agent cannot weaken the definition of correct to make its own work pass.
- Assert on observable behaviour rather than internal structure, so assertions survive refactoring.

Contract assertions versus after-the-fact tests

The same feature, specified two ways. The upper form is written before implementation and constrains it; the lower form is written afterwards and merely describes whatever was built.

```
# validation contract, written during planning
A8 A message posted to a channel appears for every current member
   of that channel without a page reload.
A9 A message posted to a channel is NOT visible to a user who is
   not a member of it, including via direct API access.
A10 Message history survives a server restart.

# test written after the code, by the agent that wrote the code
def test_post_message():
    svc = MessageService(mock_broadcaster)
    svc.post(channel_id=1, body="hi")
    assert svc.repo.saved                # an implementation detail
    assert mock_broadcaster.called      # proves a call, not an outcome
# passes forever, even if nobody ever receives the message
```


Assertion coverage as a build gate

Because assertions are assigned to features, you can check the contract is fully allocated before a single line is written — a planning-time error rather than a discovery three days in.

```
contract    = set(range(1, 148))                # A1..A147
covered     = set().union(*(f.asserts for f in features))

assert covered == contract, (
    f'unallocated assertions: {sorted(contract - covered)}'
)
```

05 Two validators, adversarial by design

6:57

A scrutiny validator runs the suite and spawns per-feature code review; a user-testing validator drives the live application like a QA engineer.

After each milestone of features, two kinds of validator run.

The **scrutiny validator** is the more traditional one. It runs the test suite, type checking, and lints, and — critically — spawns dedicated code review agents for each completed feature in that milestone. One reviewer per feature, rather than one reviewer squinting at an enormous diff.

The **user-testing validator** is the more interesting one, and it behaves like a QA engineer. It spawns the application and interacts with it through computer use or something similar: filling out forms, checking that pages render correctly, clicking buttons, and confirming that functional flows work holistically. This is what 'validate behaviour, not code' means concretely.

That step takes significantly longer than scrutiny, because the system is driving a live application rather than reading text. The consequence is counter-intuitive and worth internalising for capacity planning: **most of a mission's wall-clock time is spent waiting for real-world execution, not generating tokens.** Your throughput is bounded by how fast the application under test responds, so time and token spend decouple.

The property that makes both validators trustworthy is that neither has seen the code before. They are not invested in the implementation and have no stake in its succeeding — validation is adversarial by design. This is creator-verifier applied rigorously: fresh context is not an optimisation here, it is the entire source of the validator's credibility.

KEY POINTS

- Scrutiny validator: test suite, type checking, lints, plus a dedicated code review agent per completed feature.
- User-testing validator: spawns the app and drives it via computer use — forms, rendering, clicks, whole flows.
- Behavioural validation takes far longer than static checks because it waits on a live application.
- Most of a mission's wall-clock time goes to real-world execution rather than token generation, so time and cost decouple.
- Neither validator has seen the code before, so validation is adversarial by design — fresh context is the source of its credibility.

What each validator catches that the other cannot

The two are complementary, and the second one is the reason a multi-day run does not drift.

scrutiny validator	user-testing validator
-----	-----
type errors	a button that renders but
lint violations	does nothing when clicked
failing unit tests	a form that accepts input and
dead code, bad patterns	silently drops it
per-feature review comments	a flow that works step by step
	but breaks end to end
fast, cheap, reads text	slow, expensive, drives the app

06 Structured handoffs and self-healing

8:19

A worker does not just report 'done' — it writes down what happened, and the system corrects course at milestone boundaries.

For a system running many days, catching bugs is not enough; context must survive between agents. When a worker finishes a feature it fills out a structured handoff rather than declaring itself done.

The handoff records what was completed, what was left undone, which commands were run during that agent loop and what their exit codes were, what issues were discovered, and whether the worker abided by the procedures the orchestrator defined for it. That is a deliberately specific list, and each field earns its place: 'left undone' prevents silent partial work, exit codes give the next agent ground truth instead of a summary, and the procedures check surfaces an agent that quietly went its own way.

This is how the system self-heals. Errors get caught at milestone boundaries, corrective work is scoped into follow-up features, and the mission pulls itself back on track — as the talk puts it, not by hoping that agents remember what happened, but by forcing them to write it down and then actually address the issues. Progress is blocked while handoff issues remain unaddressed, so problems cannot be skipped past.

The longest mission run so far lasted sixteen days, longer than a full sprint, and thirty is thought feasible. That is only possible because of this structure: durable written state is what makes a run's length independent of any single agent's memory.

KEY POINTS

- A handoff records: completed work, work left undone, commands run with their exit codes, issues discovered, and procedure compliance.
- Exit codes give the next agent ground truth rather than a prose summary that may be wrong.
- Errors surface at milestone boundaries, where corrective work is scoped into follow-up features.
- Progress is blocked while handoff issues are unaddressed, so nothing gets skipped.
- The longest run was 16 days, with 30 believed feasible — durable written state is what makes that possible.

A structured handoff

Every field is actionable by the next agent or by the orchestrator deciding whether to rescope. Contrast it with 'Implemented the invite flow. All tests pass.'

```
feature: F3  invite by email
status:  partial

completed:
  - POST /invites creates a pending invite row
  - invite email renders and sends via the mailer
left_undone:
  - expiry of stale invites (A5) - needs a scheduled job,
    no scheduler exists in this repo yet
commands_run:
  - pytest tests/test_invites.py          exit 0
  - npm run typecheck                     exit 0
  - npm run e2e -- invites                 exit 1 (flaky selector)
issues_discovered:
  - mailer has no retry; a bounce loses the invite silently
  - A5 cannot be satisfied without infrastructure work
procedures_followed: yes, except "no new deps" - added croniter
asserts_claimed: A4
asserts_not_met:  A5
```

How that becomes corrective work

At the milestone boundary the orchestrator reads the handoff and negotiates the plan back into shape — this is the negotiation pattern in practice.

orchestrator at milestone 1 boundary:

```
A5 unmet          -> new feature F3b "invite expiry + scheduler"
mailer retry gap  -> new feature F3c "mailer retry + bounce log"
croniter dep      -> procedure violation, flag for human review
flaky e2e         -> not a product bug; fix the selector in F3b
```

milestone 1 stays OPEN until F3b and F3c pass validation

07 Serial execution beats parallelism

9:00

Ten agents did not give ten times the throughput — they conflicted, duplicated work, and made inconsistent decisions until coordination overhead ate the gains.

The obvious way to go faster is parallelism: ten agents at once should mean ten times the throughput. They tried it, and for software development it does not work.

The failures are specific. Agents step on each other's changes. They duplicate work neither knows the other is doing. And most damagingly they make inconsistent architectural decisions, because each is choosing in ignorance of the others. The coordination overhead required to prevent all this ends up consuming the speed gains, and you burn tokens throughout.

So missions runs features **serially** — only one worker or validator active at any moment. Parallelism is retained where it is safe: within a feature, read-only operations such as searching the codebase or researching APIs run in parallel, and within validators, read-only work such as code review is parallelised across features. The shape is serial execution with targeted internal parallelisation.

On paper this is slower. In practice the error rate drops dramatically, and for tasks running over many days that correctness compounds — every avoided inconsistency is one that does not have to be discovered and unpicked later. The general principle worth extracting: parallelise reads freely, serialise writes. Conflicts come from concurrent mutation, so the operations that only observe are exactly the ones that are safe to fan out.

KEY POINTS

- Parallel workers conflict: they overwrite each other, duplicate work, and make inconsistent architectural decisions.
- Coordination overhead consumes the speed gains while tokens are still being burned.
- Missions runs features serially — one worker or validator at a time.
- Read-only operations are parallelised inside a feature (codebase search, API research) and inside validators (code review).
- Slower on paper, but the error rate falls sharply and correctness compounds across a multi-day run.
- The transferable rule: parallelise reads, serialise writes.

Where parallelism is safe and where it is not

The dividing line is mutation. Anything that only reads can fan out; anything that writes to the shared codebase takes its turn.

SERIAL (one at a time, mutates shared state)

- worker implementing F4
- worker implementing F5
- validator applying fixes

PARALLEL (read-only, safe to fan out)

- grep/search across the codebase
- reading API docs for three candidate libraries
- code review of F1, F2, F3 simultaneously
- type check + lint + test suite

The inconsistency that costs the most

Two workers in parallel both need caching. One reaches for Redis, the other for an in-process LRU. Both features pass their own assertions. You now have two caching strategies, a dependency nobody agreed on, and a cache-coherence bug that will surface days later in a flow neither worker touched — a class of failure serial execution simply prevents.

08 The right model in each seat

11:03

Planning wants slow careful reasoning, implementation wants fast code fluency, validation wants precise instruction following — and no one provider is best at all three.

Everything above assumes you are using the right model in each role, because the three roles reward genuinely different strengths. Planning benefits from slow, careful reasoning. Implementation benefits from fast code fluency and creativity. Validation benefits from precise instruction following. No single model — and no single provider — is best at all three.

Operating a system like this therefore demands a new skill, which the team calls 'droid whispering': mentally modelling how different models behave, where each fails, how those failures compound over a multi-day run, and then deliberately choosing which model sits in which seat. Their engineer Theo produced the defaults from building the prototype, but teams are encouraged to customise them to their project.

One configuration choice deserves highlighting: validation may use a **different provider entirely**, so the verifier is not biased by the same training data as the implementer. That is creator-verifier taken to its logical end — you want the checker's blind spots to be different from the builder's, not merely its context.

This is the structural argument for a model-agnostic architecture: you are only as strong as your weakest link, and if you are locked into one provider you are constrained by that family's weakest capability. As models specialise, putting the right model in the right seat compounds. The relationship also runs the other way — validation contracts and milestone checkpoints supply enough structure that missions run successfully even on open-weight models that are not at frontier performance. Structure substitutes for raw capability.

KEY POINTS

- Planning rewards slow careful reasoning; implementation rewards fast code fluency and creativity; validation rewards precise instruction following.
- No single model or provider is best at all three, so the seats should be filled independently.
- Use a different provider for validation so the verifier is not biased by the implementer's training data.
- Locked into one provider, you inherit that family's weakest capability as your ceiling.
- Structure substitutes for capability: contracts and checkpoints let missions succeed even with open-weight models.

A model-per-seat configuration

Each seat is chosen for the property that role depends on, with the validator deliberately drawn from a different provider than the worker.

```
roles:
  orchestrator:
    model: <strongest reasoning model available>
    why: plans, decomposes, writes the validation contract
    notes: slow and expensive is fine - it runs once per mission

  worker:
    model: <fast, fluent coding model>
    why: writes most of the tokens and most of the code
    notes: cost matters here; this seat dominates spend

  validator:
    model: <precise instruction-follower, DIFFERENT provider>
    why: must apply the contract literally, not creatively
    notes: different provider => different blind spots
           than the worker that wrote the code
```

Failures compounding over a long run

A model that is slightly too agreeable is harmless in a single session and corrosive in the validator seat over sixteen days: each lenient pass admits a small defect, later features build on those defects, and the drift is only discovered when a behavioural assertion finally fails in a flow far from the original cause. Choosing seats means predicting which failure mode compounds.

09 Beating the bitter lesson: prompts, not state machines

14:27

Keep orchestration logic in text so the next model release improves the system instead of obsoleting it.

Everyone building multi-agent systems fears that the next model release will make their architecture obsolete overnight — the bitter lesson, which holds that general methods leveraging more compute beat hand-engineered structure. The design response was to require that the system get *better* with every model improvement.

Concretely, almost all orchestration logic lives in prompts and skills rather than a hard-coded state machine. How the system decomposes features and handles failures is expressed in roughly 700 lines of text, and four sentences of it can alter the execution strategy dramatically. Worker behaviour is driven by skills that the orchestrator defines per mission, so behaviour is customised per project rather than compiled in.

The only deterministic logic is deliberately thin, and it is focused on bookkeeping: running validation, and blocking progress when handoff issues have not been addressed. The division is clean — **missions ensure the discipline, models provide the intelligence** — and it is built on primitives models already understand, like AGENTS.md and skills.

Why this survives a model upgrade: a state machine encodes what today's models cannot do, so improvements make its branches dead weight. Prompt-expressed intent encodes what you want, so a more capable model simply executes it better. The deterministic layer, being pure bookkeeping, is orthogonal to capability and does not need revisiting either.

KEY POINTS

- The fear: a new model release obsoletes your architecture. The requirement: the system must improve with every model improvement.
- Almost all orchestration lives in prompts and skills — roughly 700 lines of text — not a hard-coded state machine.
- Four sentences of that text can substantially change the execution strategy.
- Worker behaviour comes from skills the orchestrator defines per mission, so it is customised per project.
- The thin deterministic layer only does bookkeeping: run validation, block progress on unaddressed handoffs.
- Missions supply the discipline; models supply the intelligence — built on primitives like AGENTS.md and skills.

State machine versus prompt-expressed policy

The same failure-handling policy, encoded two ways. The lower form gets better when the model gets better; the upper form has to be rewritten.

```
# hard-coded: enumerates cases a weaker model could not judge
if result.exit_code != 0:
    if "timeout" in result.stderr:      retry(max=3)
    elif "conflict" in result.stderr:  rebase_and_retry()
    elif result.attempts > 2:          escalate_to_human()
    else:                              spawn_fix_worker()

# prompt-expressed: states intent, improves with the model
"""
When a worker's handoff reports unmet assertions or non-zero exit
codes, decide whether the cause is environmental (retry), a scoping
error (create a follow-up feature), or a plan error (rescope the
milestone). Prefer creating a follow-up feature over retrying more
than once. Never close a milestone with unmet assertions.
"""
```

10 Production reality, and what it unlocks

13:06

A Slack clone built end to end: 60% of time and tokens on implementation, validation

never passing first time, and half the final codebase being tests.

The numbers from building a Slack clone make the architecture concrete. Implementation consumed about 60% of both time and tokens. Validation **never succeeded on the first attempt** — follow-up features were almost always required, which is the clearest evidence that the QA loop is doing real work rather than rubber-stamping. The finished codebase was around 50% test code by line count, with roughly 90% coverage. Prompt caching was used heavily to offset the cost of such long runs.

Because a multi-day run cannot be supervised through a chat transcript, there is a dedicated view — mission control — showing how much of the project is complete, how much of the budget has been burned, what the active worker is doing right now, and the handoff summaries describing what validators discovered and how the plan is changing. That lets you either act as a project manager overseeing implementation, or step away entirely.

In production the enterprise uses cluster around prototyping features overnight, building internal tools quickly, running large refactors and migrations, ML research, and modernising codebases specifically so that agents are more productive in them.

The economics are what the whole talk builds toward. A team of five engineers that previously ran about ten work streams can run perhaps thirty, with the humans focused on architecture and product decisions rather than execution. And the codebase ends up *cleaner* than it started: the end-to-end tests, unit tests, skills, and structure that missions produce leave both agents and humans more productive afterwards — which is the opposite of the usual outcome when you optimise for speed.

KEY POINTS

- Slack clone: ~60% of both wall-clock time and tokens went to implementation.
- Validation never passed on the first attempt; follow-up features were nearly always needed.
- The final codebase was ~50% tests by line count with ~90% coverage.
- Prompt caching is used heavily to offset the cost of multi-day runs.
- Mission control shows progress, budget burned, the active worker, and handoff summaries — a chat log cannot serve this.
- Enterprise uses: overnight prototypes, internal tools, large refactors and migrations, ML research, codebase modernisation.
- Five engineers go from ~10 work streams to ~30, and the codebase ends up cleaner than it started.

Reading the production numbers

Each figure implies a design consequence, which is the useful way to read them.

60% time + 60% tokens on implementation

-> the worker seat dominates spend; optimise its model choice

validation never passes first time

-> budget for follow-up features; a plan is a hypothesis

50% of final lines are tests, ~90% coverage

-> the contract forces real tests, not coverage theatre

most wall-clock time waiting on the live app

-> throughput is bound by the app, not the model;
time and token cost decouple

heavy prompt caching

-> long runs re-send large stable prefixes; cache them

Mapping the five patterns back onto the system

Where each primitive from the taxonomy actually shows up once the architecture is assembled.

delegation	orchestrator spawns workers; workers spawn research sub-agents
creator-verifier	implementation and validation are always separate agents with separate context
broadcast	shared mission state that every agent reads
negotiation	milestone boundaries: is this handoff correct, do we add follow-up features, do we rescope?
direct comms	deliberately absent - it fragments state

Glossary

Mission

A long-running multi-agent workflow: you approve a plan, then an ecosystem of agents executes it for hours or days via structured handoffs and shared state.

Delegation

One agent spawning another to do a scoped piece of work and return a result — the simplest multi-agent pattern.

Creator-verifier

Separating the agent that builds something from the agent that checks it, because the author is biased toward its own work succeeding.

Direct communication

Agents messaging each other with no central coordinator. Avoided here because state fragments and no single source of truth exists.

Negotiation

Agents coordinating over a shared resource such as an API or a region of the codebase, ideally finding a positive-sum outcome.

Broadcast

One agent distributing information to many — status, new context, shared constraints — to keep a long run coherent.

Orchestrator

The planning role: interrogates requirements and produces features, milestones, and the validation contract.

Worker

The implementation role: takes one feature with clean context, implements it, and commits so the next worker inherits a working codebase.

Validator

The verification role: an agent that has never seen the code, checking it against the contract — adversarial by design.

Validation contract

A set of assertions defining correctness, written during planning before any code, with each feature assigned the assertions it must satisfy.

Scrutiny validator

The static validator: test suite, type checking, lints, plus a dedicated code review agent per completed feature.

User testing validator

The behavioural validator: spawns the running application and drives it via computer use, checking whole functional flows.

Structured handoff

A worker's written record of what it completed, what it left undone, the commands and exit codes, issues found, and whether it followed procedure.

Milestone boundary

The checkpoint after a group of features where validators run, handoffs are reviewed, and corrective work is scoped.

Droid whispering

The skill of modelling how different models behave and fail, and deliberately assigning each to the role it suits.

Bitter lesson

The observation that general methods leveraging more compute tend to beat hand-engineered structure — hence keeping orchestration in prompts rather than code.

Check yourself

Answer first, then open each one.

Why does an agent's own test suite passing with high coverage fail to establish that its feature is correct?

Because tests written after the implementation are shaped by the code rather than by the intent — they assert what the code happens to do, so they confirm decisions instead of catching bugs. A validation contract written before any code breaks the circularity, since the definition of correct cannot be adjusted to fit whatever was built.

Ten parallel workers should give roughly ten times the throughput. Why did serial execution win instead?

Because parallel workers mutate shared state: they overwrite each other's changes, duplicate work, and make architectural decisions in ignorance of one another. The coordination overhead needed to prevent that ate the speed gains while tokens were still being spent, and the inconsistencies had to be unpicked later. Serialising writes while parallelising read-only work keeps the speed that is safe to have.

Why is it significant that neither validator has seen the code before, and why might the validator use a different model provider than the worker?

Fresh context is what makes the validator credible: an agent that wrote the code has a stake in it working and will rationalise its own choices. Using a different provider extends that independence from context to capability — a model trained differently has different blind spots, so it is less likely to accept the same mistakes the implementer was prone to making.

A worker reports 'done, all tests pass'. What does a structured handoff add, and how does the system use it?

It adds what was left undone, the actual commands with exit codes, issues discovered, and whether the worker followed the defined procedures — ground truth rather than a summary that may be optimistic. At the milestone boundary the orchestrator reads it and scopes corrective work into follow-up features, and progress stays blocked while issues are unaddressed, which is how the system self-heals instead of drifting.

Why does keeping orchestration logic in about 700 lines of prompt text, rather than a state machine, make the system resistant to the bitter lesson?

Because a state machine encodes the cases today's models cannot handle, so each model improvement turns more of its branches into dead weight that must be rewritten. Prompt-expressed policy states the intended outcome, so a more capable model simply carries it out better. The thin deterministic layer that remains is pure bookkeeping, which is orthogonal to model capability.

Most of a mission's wall-clock time is spent waiting on real-world execution rather than generating tokens. What follows from that for planning a run?

That elapsed time and token cost decouple, so you cannot estimate one from the other. Throughput becomes bounded by how fast the application under test responds, which means speeding up the model buys less than expected, while making the app faster to exercise — or parallelising read-only validation — buys more.

Where to go next

- Write a validation contract for a feature you have already built: state correctness as observable assertions, then check whether your existing tests actually verify them.
- Add a verifier step to an agent workflow you own, with strictly fresh context, and measure how many issues it finds that the implementing agent's own tests missed.
- Replace a 'done' message in your pipeline with a structured handoff including commands and exit codes, and see what it surfaces that prose summaries hid.
- Take a parallel agent setup and classify every operation as read or write; serialise the writes and fan out the reads, then compare error rates.
- Try a different provider in the validator seat than in the worker seat and look for defects the same-family validator was consistently waving through.
- Find the hard-coded branches in your own orchestration that exist only because models used to be weaker, and try expressing that policy as prompt text instead.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.